

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 4

Operating Systems

Topic: The xv6 Shell from the Outside

bash versus sh, argv, and the Shell Source

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Fixing sleep for the current xv6 **4**

2 bash versus the xv6 shell **6**

3 Who expands ∗? A program that prints its argv **9**

4 Reading the shell **12**

Conclusions **15**

INTRODUCTION

The whole user interface of xv6 is a single program, `user/sh.c`, about 500 lines long. It reads a line, splits it into a command and its arguments, and then either runs the program or does the small amount of work that a program cannot do for itself. In this laboratory work that shell is used side by side with bash: the same commands are typed in both, and the differences show what a shell must do and what bash adds on top of it.

The work has four parts. First the `sleep` program from Laboratory Work No. 2 is fixed for the current xv6, where the system call was renamed from `sleep` to `pause`. Then ten commands are run in bash and in the xv6 shell, and the results are collected in a table. Third, a tiny program, `args`, is written twice, once for xv6 and once for Linux, to see exactly which arguments each shell hands to a program and therefore who expands `*` and who removes quotes. Finally, four `grep` commands are run over `user/sh.c` to read how the shell is built from the inside.

I used the Linux virtual machine of the course with the same `xv6-riscv` tree as in the previous labs. The submitted files are `user/sleep.c`, `user/args.c`, `args_linux.c` and the `Makefile`.

1 FIXING SLEEP FOR THE CURRENT XV6

In August 2025 the xv6 authors renamed the `sleep` system call to `pause`, so a `sleep` program written for an older tree no longer compiles. The first step is to ask the tree which name it uses:

```
$ grep -n "pause\|sleep" user/user.h
int pause(int);
```

Listing 1: Checking the name of the system call in user/user.h

missing sc-1.png

Figure 1: The grep result in user/user.h

My tree declares `pause`, so the call inside the program has to change. The program itself is still called `sleep`, because that is the name the course gives to it and the name that must appear in `UPROGS`.

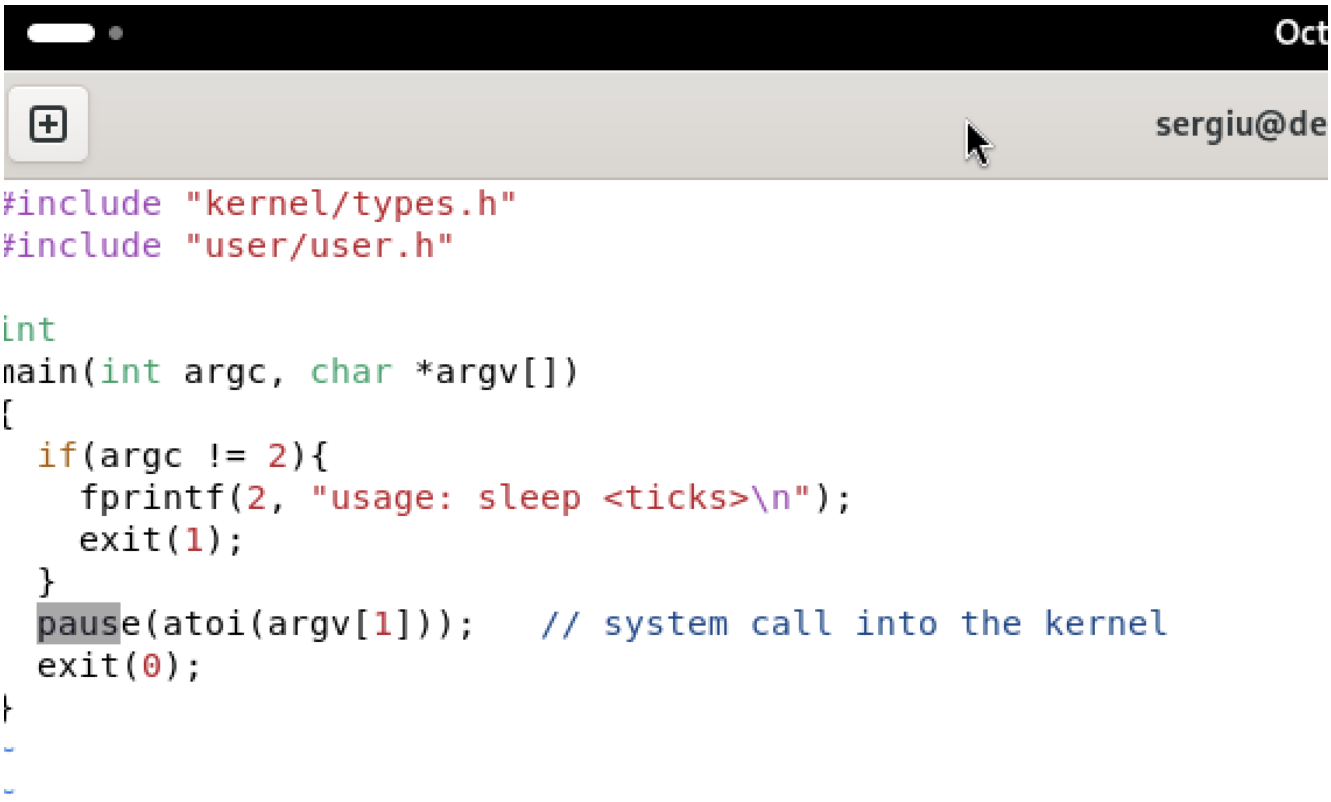


Figure 2: The updated user/sleep.c

```
1 #include "kernel/types.h"
2 #include "user/user.h"
3
```

```
int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before August 2025
  exit(0);
}
```

Listing 2: user/sleep.c after the change

Only one line changes: the comment marks the call that used to be `sleep()`. The rest of the program works as before, since the two names describe the same system call with the same argument, a number of clock ticks.

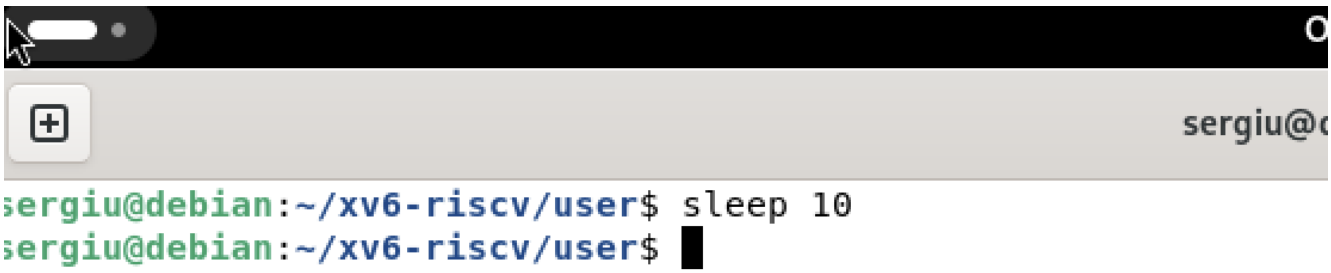


Figure 3: Running the program with an argument

After `make qemu` the program is rebuilt into `fs.img` and starts from the xv6 prompt. With one argument it waits for that many ticks and returns to the prompt without printing anything. Without an argument `argc` is 1, so the program prints `usage: sleep <ticks>` and exits with status 1 instead of reading a missing argument.

2 BASH VERSUS THE XV6 SHELL

Every command was run twice: once in bash on Linux, inside an empty folder that contains a file named `README`, and once at the xv6 prompt.

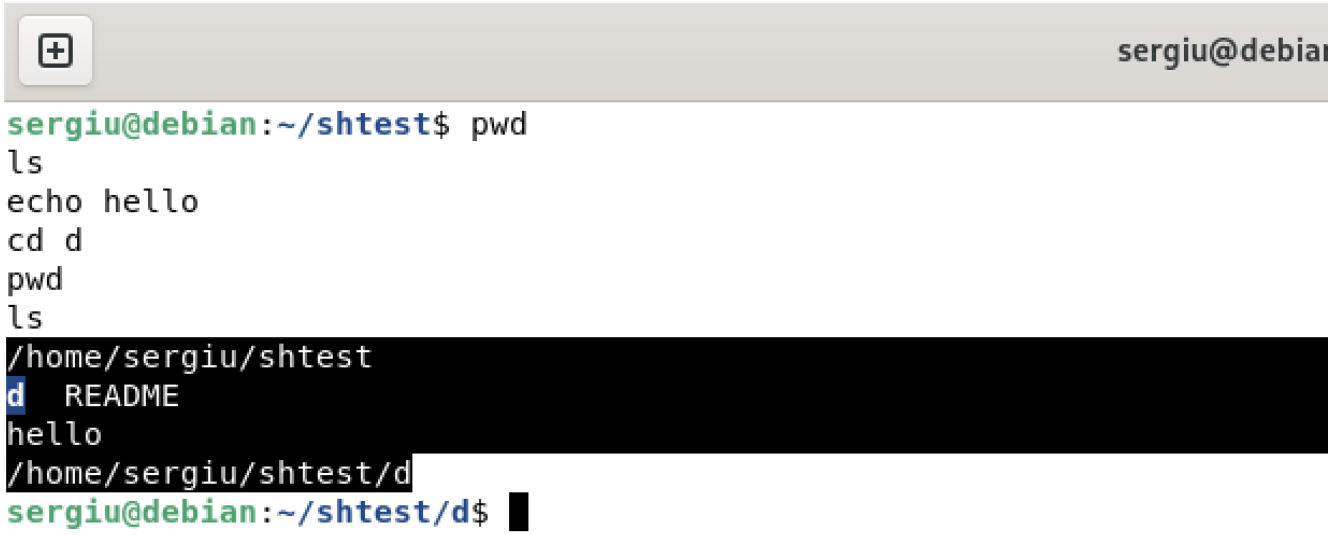


Figure 4: Basic commands and cd in bash

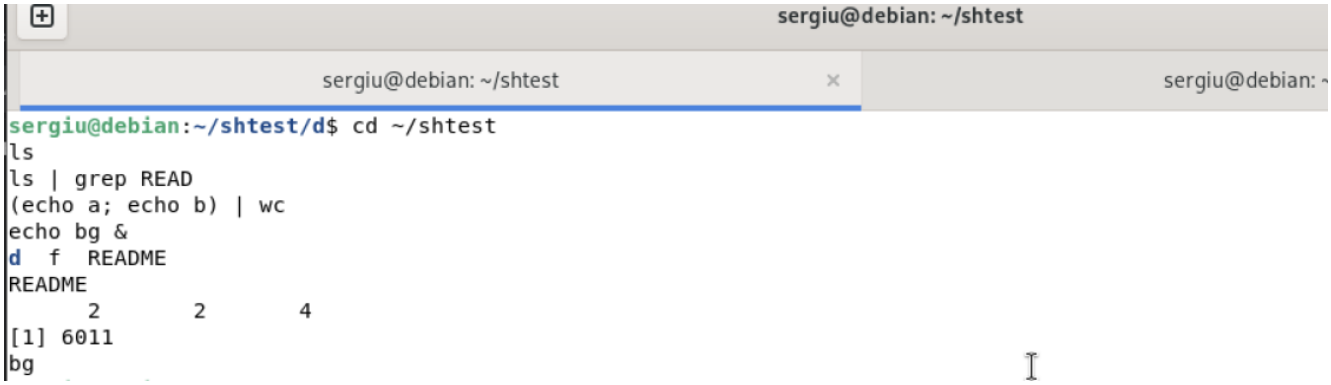


Figure 5: Redirection, pipe, grouping and background in bash

In bash `pwd` prints the current directory, `ls` lists it and `echo` prints its arguments. A pipe connects the standard output of one program to the standard input of the next, so `ls | grep READ` keeps only the line with `README`, and `(echo a; echo b) | wc` counts the output of a group of commands. A trailing `&` puts the command in the background and bash prints a job number.

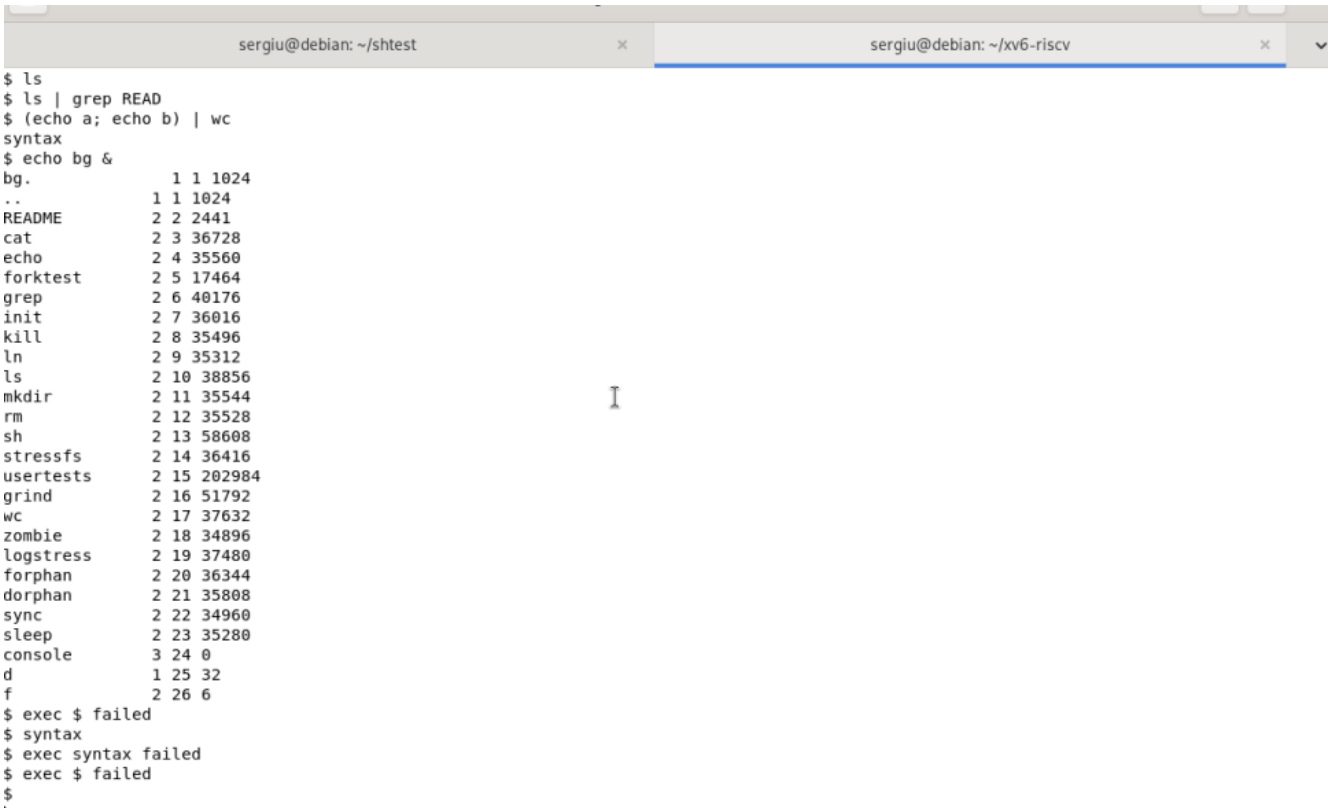


Figure 6: The same commands in xv6



Figure 7: ls failing after cd d, and the working alternative /ls

The xv6 shell runs the same programs, but it does much less around them. Its `ls` prints the name, the type, the inode number and the size of every entry, so `ls | grep READ` shows more than bash. There is no globbing, no variable expansion and no quote removal, and the shell cannot read a command that continues on a second line.

Command	Tests	bash	xv6
`echo hello > f ; cat < f`	redirection	works	works
`ls | grep READ`	pipe	prints `README`	works, but prints name, type, inode and size
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`syntax`: no multi-line commands
`echo one ; echo two`	sequence	`one`, then `two`	`one`, then `two`
`echo bg &`	background	`[1] 6011` and `bg`	`bg`
`ls *`	globbing	`f README`	`ls: cannot open *`
`echo $HOME`	variables	`/home/sergiu`	`$HOME`
`echo "a b"`	quotes	`a b`	`"a b"`
`mkdir d ; cd d ; ls ..`	PATH	`d f README`	`exec mkdir failed`, `exec cd failed`
arrow up, Tab, Ctrl+R	history, completion	work	do not work

Table 1: The same commands in both shells.

1. List the features both shells have, and the ones only bash has.

Both shells execute programs, connect them with pipes (`|`), redirect input and output (`<`, `>`), run a command in the background (`&`), group commands with parentheses, run commands in sequence with `;`, and change directory with `cd`.

Only bash has command history, tab completion, environment variables and `$` expansion, a `PATH` to look programs up, job control (`jobs`, `fg`, `bg`), command substitution, and a scripting language with variables and loops.

2. In xv6, `ls` stops working after `cd d`. Why? Find a way to run `ls` from inside `d` anyway.

Because xv6 has no `PATH` variable. When a name contains no slash, `exec` looks for the file in the current directory only. After `cd d` the current directory is the empty `d`, so there is no file called `ls` to run. Giving the absolute path `/ls` works, because a name that starts with a slash is resolved from the root of the file system, where the programs of `fs.img` actually live.

3. Which missing feature would you miss most in daily work, and which would be easiest to add?

Command history and tab completion, because they are used in every single command. History would also be the easiest to add: the shell already reads its input with `getcmd`, so it could keep the lines it has read in a buffer and let the arrow keys walk through them.

3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

The same program is written twice. For xv6 it uses the two xv6 headers and `exit`, and it is added to `UPROGS` like `sleep`. For Linux only `stdio.h` is needed and `main` returns 0.

```c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for (int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

Listing 3: user/args.c for xv6

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for (int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

Listing 4: args_linux.c for Linux

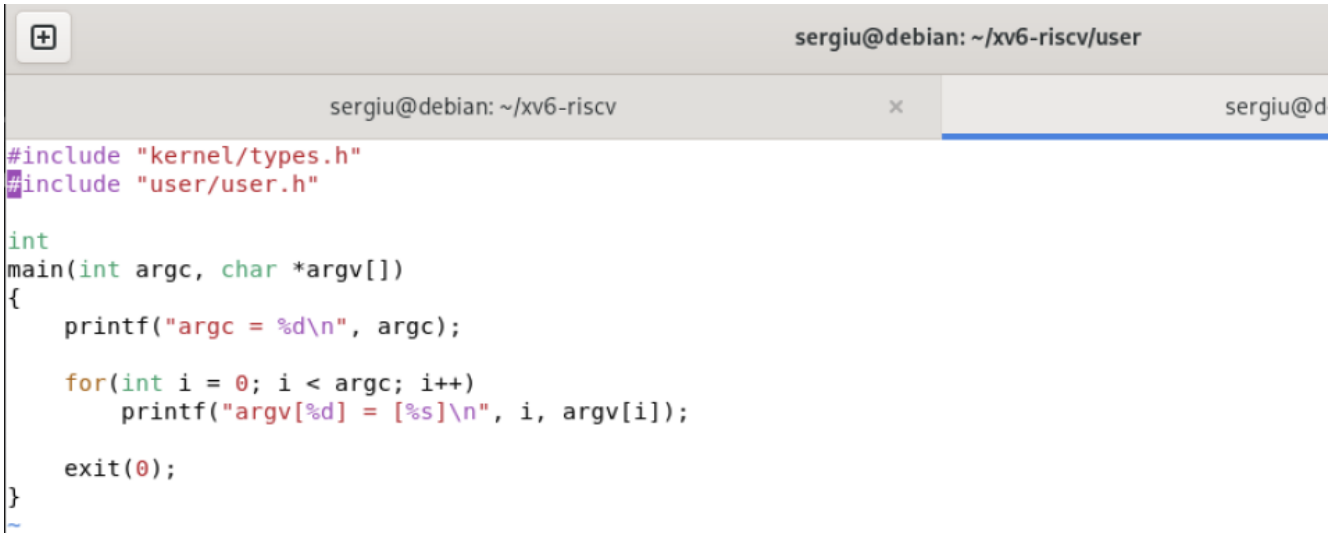


Figure 8: args.c and the UPROGS line

The Linux version is compiled with `gcc -o args args_linux.c` and started as `./args`, because the current directory is not in `PATH` by default. The xv6 version is started as `args`, because in xv6 every program lives in the directory the shell is in.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ args one two three
$ args *.c "x y"
$ args $HOME
$ args 'single quoted'argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ exec $ failed
$ exec $ failed
$
```

Figure 9: The args output in xv6

```
sergiu@debian: ~/shtest                    sergiu@debian: ~/shtest
sergiu@debian:~$ cd ~/shtest
sergiu@debian:~/shtest$ vim args_linux.c
sergiu@debian:~/shtest$ gcc -o args args_linux.c
sergiu@debian:~/shtest$ ./args one two three
./args *.c "x y"
./args $HOME
./args 'single quoted'
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
argc = 2
argv[0] = [./args]
argv[1] = [/home/sergiu]
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 10: The args_linux output in bash

1. Paste both outputs for `args *.c "x y"`. How many arguments did each program receive?

In bash the program receives 3 arguments:

```
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
```

In xv6 it receives 4:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

The difference is one argument: bash replaced the pattern with a file name and joined `"x y"` into a single word, while xv6 kept both the star and the quotes as ordinary characters.

2. So who turns `*.c` into file names and removes the quotes: the program, the kernel, or the shell?

The shell. The program only receives the list of words that is already prepared, and the kernel only copies that list into the new process when `exec` is called. Since the same `args` program prints a file name under bash and a literal star under xv6, the change must have happened before the program started: it is part of what the shell does when it parses the line.

3. What happened to the extra spaces in `one two three`?

They disappear before the program runs. The shell treats every run of unquoted spaces as a single separator, so `args one two three` arrives as `argv[1] = [one]`, `argv[2] = [two]`, `argv[3] = [three]`, and the program cannot tell how many spaces were typed. Both shells behave the same way here.

4 READING THE SHELL

The shell is one `while` loop in `main`: read a line with `getcmd`, parse it into a tree of commands, run the tree, repeat. Four `grep` commands show the parts of that loop.

```
sergiu@debian:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
sergiu@debian:~/xv6-riscv$
```

Figure 11: The kinds of command defined in user/sh.c

`user/sh.c` has about 500 lines. The five kinds of command are numbered constants, and each kind is represented by a struct that points to its parts: an `execcmd` holds the argument vector, a `redircmd` holds the file to open, and the pipe, list and background commands hold pointers to the commands they connect.

```
    if (fork1() == 0) {
      close(1);
      dup(p[1]);
      close(p[0]);
      close(p[1]);
      runcmd(pcmd->left);
    }
    if (fork1() == 0) {
      close(0);
      dup(p[0]);
      close(p[0]);
      close(p[1]);
      runcmd(pcmd->right);
    }
    close(p[0]);
    close(p[1]);
    wait(0);
    wait(0);
    break;

  case BACK:
    bcmd = (struct backcmd *)cmd;
    if (fork1() == 0)
      runcmd(bcmd->cmd);
    break;
  }
  exit(0);
}

int
getcmd(char *buf, int nbuf)
{
  write(2, "$ ", 2);
  memset(buf, 0, nbuf);
  gets(buf, nbuf);
  if (buf[0] == 0) // EOF
sergiu@debian:~/xv6-riscv$
```

Figure 12: Where each kind of command is run

`runcmd` is a `switch` over the kind of command. `EXEC` calls `exec`; `REDIR` opens the file,

replaces the right descriptor with `dup` and runs the inner command; `PIPE` creates a pipe, forks twice and gives each child one end; `LIST` runs the left command, waits, then the right one; `BACK` runs the left command in a child without waiting.

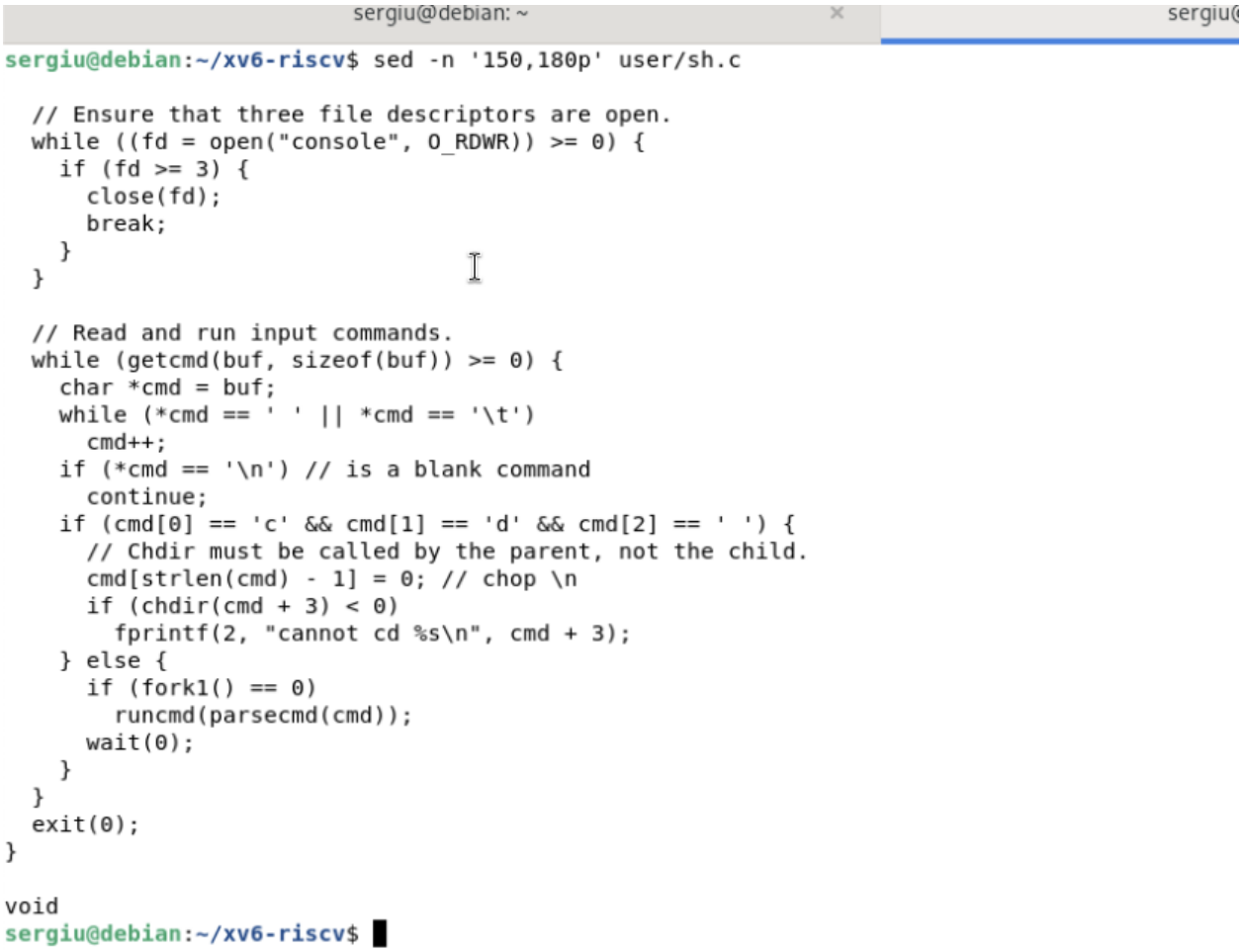


Figure 13: The code that handles cd

1. Name the five kinds of command the xv6 shell knows and the character that produces each.

`EXEC` (a plain command), `REDIR` (`<`, `>` or `»`), `PIPE` (`|`), `LIST` (`;`) and `BACK` (`&`). They are defined as the numbers 1 to 5 at the top of `user/sh.c`, together with `MAXARGS` = 10, the largest number of arguments a command may have.

2. Where is `cd` handled, and why is it not run as a separate program?

It is handled in the `main` loop of `user/sh.c`, before the line is parsed: if the line starts with `cd` followed by a space, the shell cuts the newline, calls `chdir(cmd+3)` itself and continues. A separate program could not do it, because every program runs as a child process with its own copy of the working directory. The child would change its own directory and then exit, and the shell would still be where it was. For the change to be visible, it must happen in the shell process itself.

3. Search sh.c for any code that handles `*` or `$`. What does that explain about Part 2?

There is none. The parser only knows spaces, the redirection characters, `|`, `;`, `&` and parentheses. A star and a dollar sign are copied into the argument vector like any other character, which is why `ls *` asks for a file literally named `*` and `echo $HOME` prints `$HOME`: no part of xv6 expands them.

CONCLUSIONS

This laboratory work looked at the same shell from three sides: its behaviour, its contract with programs, and its source. Running identical commands showed that xv6 and bash share the core of the Unix interface – programs, pipes, redirection, sequences, groups and background jobs – while bash adds everything that makes a shell comfortable in daily use: history, completion, variables, `PATH` lookup and a scripting language.

The `args` experiment separated the responsibilities cleanly. Wildcards and quotes never reach the program: the shell expands them while it parses the line, and the kernel only copies the finished argument list into the new process. The same program prints `*.c` in xv6 and a file name in bash, which proves that the difference is in the shell and nowhere else.

Reading `user/sh.c` explained the last mystery of the table: the file contains no code for `*` or `$`, so xv6 cannot expand them, and it handles `cd` inside its own `main` loop because a child process could not change the working directory of its parent. The shell is small enough to read completely, and after this work its behaviour stops being magic: every line of the 500-line file maps to one thing the shell does with a command line.